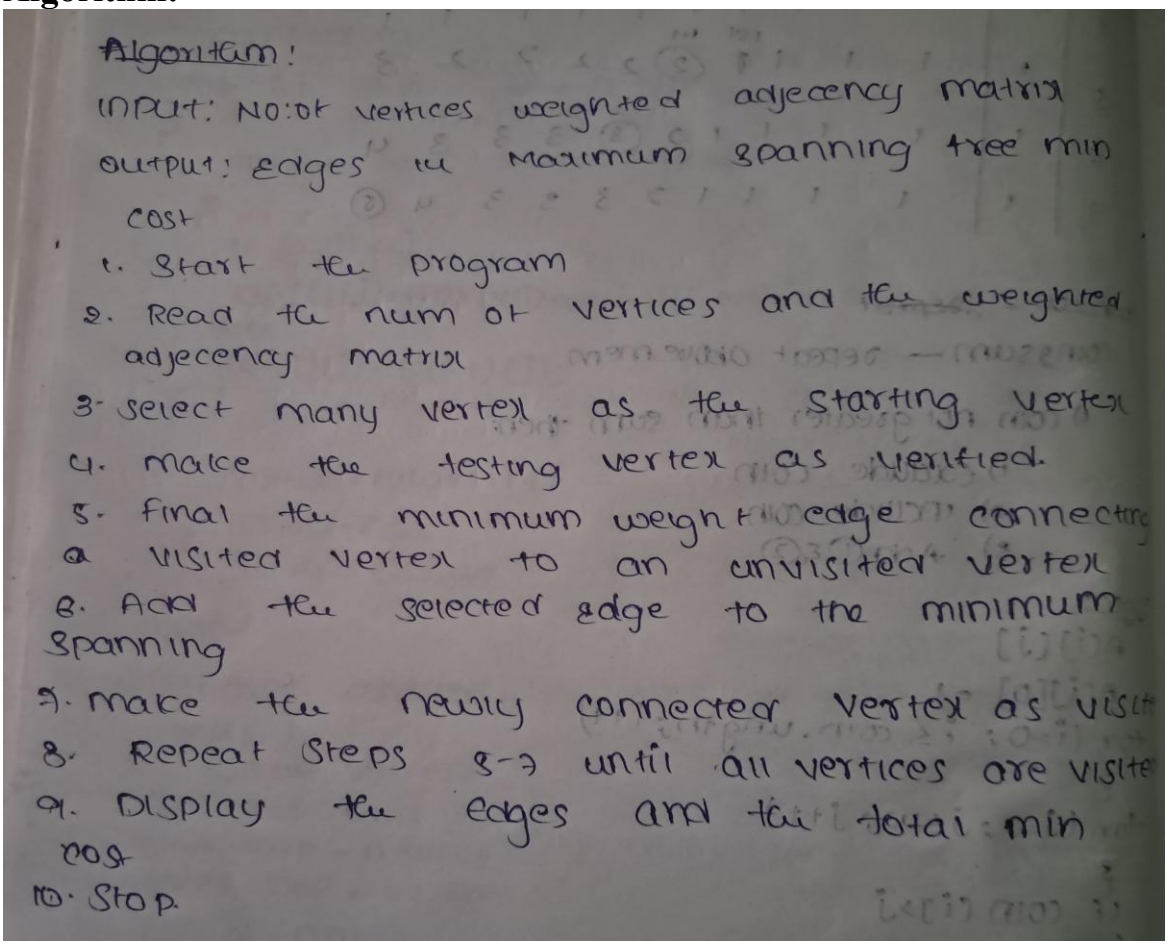
Practical –9

Aim: Implement prim's algorithm

Algorithm:



Algoritam:
INPUT: NO:of vertices weighted adjecency matrix
output: edges in Maximum spanning tree min cost
1. Start the program
2. Read the num of vertices and the weighted adjecency matrix
3. select many vertex as the starting vertex
4. make the testing vertex as verified.
5. final the minimum weight edge connecting a visited vertex to an unvisited vertex
6. Add the selected edge to the minimum spanning
7. make the newly connected vertex as visit
8. Repeat Steps 5-7 until all vertices are visite
9. Display the edges and the total min cost
10. Stop.

Program:

```
#include <iostream>

using namespace std;
```

Name: S.Jaswanth siva kumar
Enrollment no : 92460118154

```
int main() {
    int n;
    cout << "Enter number of vertices: ";
    cin >> n;

    int cost[10][10];

    cout << "Enter the adjacency matrix:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> cost[i][j];
            if (cost[i][j] == 0)
                cost[i][j] = 999;
        }
    }

    int visited[10] = {0};
    visited[0] = 1;

    int edges = 0, minCost = 0;

    cout << "\nEdges in Minimum Spanning Tree:\n";
```

```
while (edges < n - 1) {
    int min = 999, u = -1, v = -1;

    for (int i = 0; i < n; i++) {
        if (visited[i]) {
            for (int j = 0; j < n; j++) {
                if (!visited[j] && cost[i][j] < min) {
                    min = cost[i][j];
                    u = i;
                    v = j;
                }
            }
        }
    }

    if (u == -1) {
        cout << "Graph is disconnected.\n";
        return 0;
    }

    cout << u + 1 << " - " << v + 1
         << " : " << min << endl;
```

```
        visited[v] = 1;
        minCost += min;
        edges++;
    }

    cout << "Minimum Cost = " << minCost << endl;

    return 0;
}
```

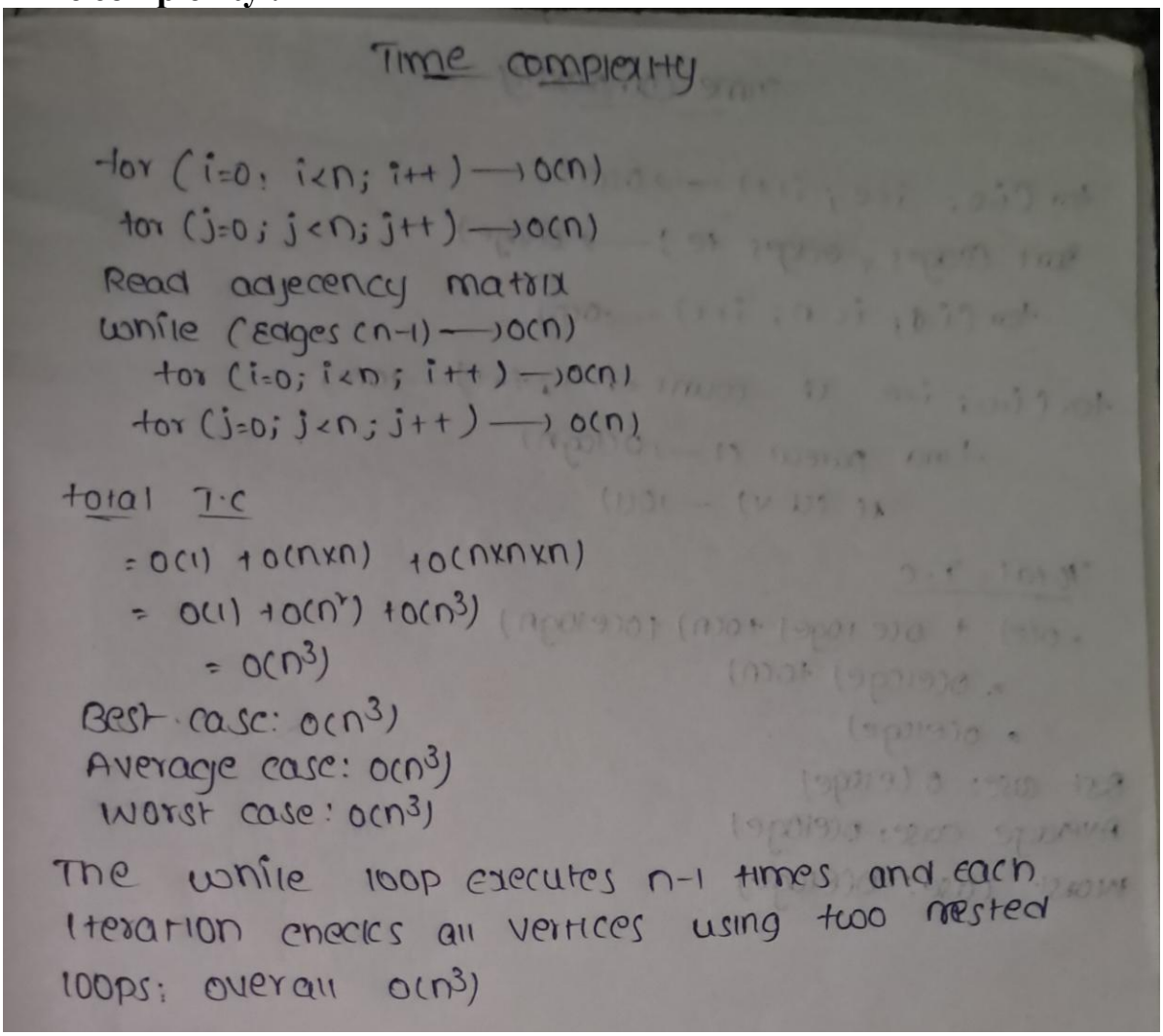
Output:

```
Enter number of vertices: 4
Enter the adjacency matrix:
0 2 0 6
2 0 3 8
0 3 0 5
6 8 5 0

Edges in Minimum Spanning Tree:
1 - 2 : 2
2 - 3 : 3
3 - 4 : 5
Minimum Cost = 10
```

Name: S.Jaswanth siva kumar
Enrollment no : 92460118154

Time complexity :



Conclusion :

Thus, we have successfully implemented Prim's algorithm in C++ to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph. The algorithm selects the minimum-weight edges without forming cycles and connects all vertices with minimum total cost. The time complexity of our code is \(O(n^3)\).

Name: S.Jaswanth siva kumar
Enrollment no : 92460118154